A Direct-Bind Impossibility for Arrow Fanout and Loop

A Mechanized Boundary for Structural Observability

Abstract

We prove a *semantic impossibility* for a precisely scoped class of encodings: no faithful *direct-bind* encoding of the arrow primitives &&& (fanout) or loop (feedback) preserves arrow semantics. A direct-bind encoding is a pair (m, k) where the arrow is encoded as $\mathsf{bind}\, m\, k$, with m and k satisfying the semantic specifications of the arrow's sub-computations, and the only communication from m to the continuation k is m's monadic result — no side channel carries the original input to k. The proof rests on a single encoding-independent law of bind (provable by rfl from bind's semantics): the continuation in $\mathsf{bind}\, m\, k$ is invoked with m's *output*, never the original arrow input. For the witness $\&\&\&(\mathsf{Get}, \mathsf{Put})$, this forces the monadic final state to be the threaded state while the arrow final state is the original input, diverging whenever *input* $\neq$ *state*—for *all* faithful direct-bind encodings, not merely the canonical translation. An analogous theorem holds for loop (with a pair/feedback value domain). Both are mechanized in Lean 4. A third result for left (choice), with a weaker hypothesis, appears in the appendix.

The result is *not* that "monads cannot encode arrows" — a monad that threads the original input through its state (e.g., a reader-state monad carrying both *input* and *state*) can encode $\&\&\&(\mathsf{Get}, \mathsf{Put})$. The result is that such an encoding is *not* a direct-bind encoding: it requires a side channel beyond m's monadic result. This sharpens the known expressiveness gap (Lindley, Wadler, Yallop (2011)) from "the inverse translation requires ArrowApply" to "no faithful direct-bind encoding of &&& or loop exists."

We formalize the information loss via an *indistinguishability* argument: $\mathsf{encodeMonad}(\&\&\&(f, g)) = \mathsf{encodeMonad}(\mathsf{seq}(f, g))$ by rfl, so any observer of the encoded monadic representation cannot distinguish fanout from sequencing. We derive a structural information ordering $\sigma(\mathsf{OM}(C)) \subseteq \sigma(\mathsf{FM}(C)) \subseteq \sigma(\mathsf{FA}(C))$, with both inclusions strict for suitable computations. ArrowApply preserves σ (the structural facts recoverable from the freer arrow representation). We do not claim maximality over all conceivable representations; a tagged representation recovers strictly more, and we prove this.

With ArrowApply, the five arrow primitives are expressively equivalent to monads (ArrowApply $\cong$ Monad, Lindley, Wadler, Yallop (2011)) but *representationally superior*: the same computational content is carried in an inspectable tree, not an opaque closure.

As a negative side-result (Appendix), the standard arrow-to-lambda translation $\mathcal{T}$ is linear ($|\mathcal{T}(p)| \leq 8 \cdot |p|$), compositional ($\mathcal{T}(f \ggg g) = \mathcal{T}(g) \circ \mathcal{T}(f)$ definitionally), and law-preserving (the loop extension law holds under $\beta\eta$ plus fix-fusion/Bekić's lemma (1969)). No translation asymmetry exists. The entire library comprises 108 Lean 4 theorems, all fully proven with zero `sorry`, supported by 242 Rust tests.

Introduction

Arrows, introduced by Hughes (2000), generalize monads to a notion of computation that need not be a strong functor. They have been proposed as a substrate for programming-language construction: a small calculus of composition, choice, and feedback from which abstraction, recursion, effects, and tooling can be *observed* rather than *postulated*.

The expressiveness gap between arrows and monads is known: Lindley, Wadler, and Yallop (2011) proved that arrows embed into monads, but monads require ArrowApply to embed back. What is *not* known is the precise *semantic* content of this gap: *which* arrow primitives cannot be encoded by monadic bind *under a direct-bind discipline*, and *why*? Prior work (2025) observed the gap empirically (freer monad continuations are opaque to static analysis); we prove a formal, scoped version.

Scope. We do not prove "monads cannot encode arrows." A monad that threads the original input through its state (a reader-state monad carrying both *input* and *state*) can encode &&(Get, Put). Our result is narrower and more precise: no *direct-bind* encoding — one where the arrow is encoded as $\mathsf{bind}\, m\, k$ with m's monadic result as the *only* communication from m to k — preserves arrow semantics for && or loop. The obstruction is structural: bind exposes only the predecessor's output to the continuation, never the original input.

Contribution. This paper makes two primary contributions, both mechanized in Lean 4:

1. **A semantic impossibility for direct-bind encodings** (Section 3). We define a formal `BindEncoding` class—a *direct-bind* encoding where the arrow is encoded as $\mathsf{bind}\, m\, k$ with no side channel carrying the original input to k—and prove that no faithful `BindEncoding` of &&(Get, Put) or $\mathsf{loop}\, \mathsf{body}$ preserves arrow semantics. The root cause is a single encoding-independent equation (provable by rfl): the continuation in $\mathsf{bind}\, m\, k$ is invoked with m's output, never the original input. The theorem `no_faithful_bind_encoding_preserves_fanout` quantifies over all `BindEncoding`s whose components satisfy Get-semantics and Put-semantics. An analogous theorem holds for loop. A third result for left (choice), with a weaker hypothesis, appears in Appendix B.
2. **A representation boundary for structural observability** (Section 4). We prove $\mathsf{encodeMonad}(\&\&(f, g)) = \mathsf{encodeMonad}(\mathsf{seq}(f, g))$ by rfl: the monadic encoding cannot distinguish fanout from sequencing. We derive a structural information ordering $\sigma(\mathsf{OM}(C)) \subseteq \sigma(\mathsf{FM}(C)) \subseteq \sigma(\mathsf{FA}(C))$ with both inclusions strict for suitable computations, prove ArrowApply preserves σ, and show that maximality over all conceivable representations is *false* (a tagged representation recovers strictly more). With ArrowApply, the freer arrow is expressively equivalent to monads (by Lindley et al.'s result (2011)) but representationally superior—the same computational content is carried in an inspectable tree, not an opaque closure.

As a negative side-result (Appendix A), the standard arrow-to-lambda translation $\mathcal{T}$ is linear, compositional, and law-preserving. No translation asymmetry exists.

Background

Arrows. An arrow (2000) is a type constructor α equipped with $\mathsf{arr} : (A \to B) \to \alpha\, A\, B$ and $\gg : \alpha\, A\, B \to \alpha\, B\, C \to \alpha\, A\, C$, satisfying category-like laws. Additional structure: first (products/fanout), left (sums/choice), loop (feedback). The arrow laws include the composition-through-first law and the loop extension law $\mathsf{loop}(f \gg \mathsf{first}\, g) = \mathsf{loop}\, f \gg g$.

Arrow calculus. Lindley, Wadler, and Yallop (2010) established a bidirectional correspondence between the arrow calculus and lambda with products and sums. Their translation is the standard one; our $\mathcal{T}$ (Appendix A) is an instance of their forward direction.

ArrowApply and monads. Lindley, Wadler, and Yallop (2011) established that ArrowApply (arrows with an app combinator for dynamic application) is isomorphic to Monad. Without app, arrows cannot express arbitrary higher-order functions; with app, they have full monadic power.

Freer monads and freer arrows. Kiselyov and Ishii (2015) introduced freer monads, which represent computations as explicit data structures. VanDomelen et al. (2025) introduced freer arrows and observed empirically that freer monad continuations are opaque to static analysis, while freer arrow trees are inspectable. They did not prove a semantic impossibility; our contribution (Section 3) is the formal, mechanized version.

Algebraic effects for arrows. Lindley (2014) generalized algebraic effects and effect handlers to support idiom and arrow computations. The strict inclusions idioms $\subset$ arrows $\subset$ monads were established by Lindley, Wadler, and Yallop (2011) via type isomorphisms. Our characterization is concrete (about specific combinators &&, left, loop) and mechanized.

A Semantic Impossibility for Direct-Bind Encodings

This is the central result. We define a formal class of direct-bind encodings, state the semantic specifications precisely, and prove that no faithful encoding of &&(Get, Put) or loop body preserves arrow semantics. The result is not "monads cannot encode arrows" — it is more precise: once the only communication channel from the first computation to the continuation is its monadic result, fanout cannot be faithfully represented when the first computation's semantics discard the original input.

The Encoding Class

To make the universal quantification precise, we define what a "direct-bind encoding" *is*.

Definition 1 (Direct-bind encoding (BindEncoding))**.** A **direct-bind encoding** of an arrow A is a pair (m, k) where: - m : Monad encodes the first sub-computation, - $k : \mathsf{V} \to \mathsf{Monad}$ is an opaque continuation encoding the second sub-computation, - the encoding of A is $\mathsf{bind}\, m\, k$.

The "no side channel" property is not a separate field of the structure but a *consequence* of the definition: since the encoding is exactly $\mathsf{bind}\, m\, k$ (no pre-processing, no duplication, no packaging of the input before the bind), the only value k receives from m is m's monadic result. The original arrow input is passed to `runMonad` as an argument but is not forwarded to k except through m's output. This is what makes the encoding "direct": the composition is a bare bind, not a bind wrapped in additional plumbing. The impossibility theorem is relative to this encoding class.

(Mechanized: `BindEncoding` structure in `FanoutUnencodable.lean`, with `BindEncoding.encoded = Monad.bind enc.m enc.k`.)

Semantic correctness—that m and k faithfully represent the arrow's sub-computations—is specified separately via semantic predicates:

Definition 2 (Get-semantics and Put-semantics)**.**

- **Get-semantics**: $\mathsf{GetSemantics}(m, \mathit{input}, \mathit{state})$ holds iff $\mathsf{runMonad}(m,\ \mathit{input},\ \mathit{state}) = (\mathit{state},\ \mathit{state})$ — the term reads the state, yields it as output, and leaves the state unchanged.
- **Put-semantics**: $\mathsf{PutSemantics}(k)$ holds iff $\forall x\, s.\ \mathsf{runMonad}(k\, x,\ x,\ s).2 = x$ — the continuation writes its input argument to the state. The monadic input is x (equal to the continuation argument), reflecting that Put's operational semantics pass the monadic input as the continuation argument. A stronger spec $\forall x\, i\, s$ (with independent i) would be too strong: Put writes the monadic input, which equals the argument by construction.

A `BindEncoding` is **faithful** to &&(Get, Put) if m satisfies Get-semantics and k satisfies Put-semantics. We restrict attention to faithful encodings — those whose components preserve the semantics of the corresponding sub-computations. The impossibility theorem is relative to this encoding class.

(Mechanized: `GetSemantics`, `PutSemantics`, `BindEncoding.faithful_fanout`.)

The Encoding-Independent Bind Law

We model monadic computations as a free monad over operations, with bind taking an *opaque continuation* (a function Value → Monad, realized as `Rc<dyn Fn(Value) -> FreeM>` in the Rust implementation).

Theorem 3 (Bind continuation law, encoding-independent)**.** *For all monadic m, continuations k, input, and state:*

$$\mathsf{runMonad}(\mathsf{bind}\, m\, k,\ \mathit{input},\ \mathit{state}) = \mathsf{let}\ (o, s') = \mathsf{runMonad}(m,\ \mathit{input},\ \mathit{state})\ \mathsf{in}\ \mathsf{runMonad}(k\, o,\ \mathit{input},\ s').$$

This is provable by rfl *from the definition of* bind*'s semantics. (Mechanized:* `bind_continuation_input_is_predecessor_outp`

Corollary 4 (Continuation cannot see original input)**.** *If m's output differs from the original input (*$\mathsf{runMonad}(m,\ \mathit{input},\ \mathit{state}).1 \neq \mathit{input}$*), then k is invoked with a value $\neq$ input. (Mechanized:* `continuation_cannot_see_original_input`*.)*

This is the root cause. Arrow combinators can preserve access to an earlier computation input across composition, whereas monadic bind exposes only the preceding computation's result to its continuation. The two theorems below — fanout and loop — are not separate impossibility mechanisms but two manifestations of this single structural fact. They form one theorem family: the bind law forces the continuation to see only the predecessor's output, and any arrow primitive that requires the original input in the continuation is subject to the same bottleneck.

Fanout: the flagship theorem

Definition 5 (The witness)**.** Let Get be the operation that reads the state and yields it as output, and Put be the operation that writes the current arrow input into the state. The witness arrow program is:

$$\mathsf{counterexample} = \&\&(\mathsf{Get},\ \mathsf{Put}).$$

Under arrow semantics, && distributes the original *input* to both branches. So Get reads the state (output = *state*), and Put writes the original *input* to the state. The arrow final state is *input*.

Theorem 6 (Arrow semantics of the witness)**.** $\mathsf{runArrow}(\&\&(\mathsf{Get}, \mathsf{Put}),\ \mathit{input},\ \mathit{state}).2 = \mathit{input}$. *(Mechanized:* `runArrow_counterexample_state`*.)*

Theorem 7 (No faithful direct-bind encoding preserves fanout)**.** *For any* `BindEncoding` (m, k) *faithful to* $\&\&(\mathsf{Get}, \mathsf{Put})$ *— i.e.,* $\mathsf{GetSemantics}(m, \mathit{input}, \mathit{state})$ *and* $\mathsf{PutSemantics}(k)$ *— if* $\mathit{input} \neq \mathit{state}$*:*

$$\mathsf{runArrow}(\&\&(\mathsf{Get}, \mathsf{Put}),\ \mathit{input},\ \mathit{state}).2 \neq \mathsf{runMonad}(\mathsf{bind}\, m\, k,\ \mathit{input},\ \mathit{state}).2.$$

(Mechanized: `no_faithful_bind_encoding_preserves_fanout`*.)*

Proof. By Theorem 1, k is invoked with m's output. By Get-semantics, m's output is *state*. By Put-semantics, k writes its argument to the state, so the monadic final state is *state*. By Theorem 2, the arrow final state is *input*. If $\mathit{input} \neq \mathit{state}$, they diverge. The canonical `arrowToMonad` is one faithful instance (verified by `arrowToMonad_is_faithful`), but the theorem holds for the entire class. □

Remark 8 (Why the witness works)**.** Get's output is constrained to exactly *state* by Get-semantics — so m cannot encode information about the original *input* into its output. bind threads this output (the state) into Put, which writes the state. The arrow, by contrast, forwards the original input to Put, which writes the input. The divergence is forced by Get-semantics (output = *state*) plus the bind law (continuation sees the output), and is independent of the encoding.

Remark 9 (What the theorem does and does not say)**.** A reader might object: "If you forbid passing the input to the continuation, of course you can't encode a combinator that requires the input." This is correct, and the theorem is deliberately scoped. The contribution is not the *intuition* (which is straightforward) but the *formalization*: a precisely defined encoding class (`BindEncoding`), precisely stated semantic specs (Get/Put-semantics), and a mechanized proof that the divergence holds for *all* faithful instances of the class, not merely the canonical translation. The theorem does not say monads cannot encode && — a reader-state monad carrying both *input* and *state* can. It says such an encoding is not a direct-bind encoding: it requires a side channel (the input threaded through the state) beyond m's monadic result.

Loop: an independent witness of the same invariant

We extend the value domain to include pairs and define Arrow_3 with a loop combinator that feeds the body *both* the input and the feedback channel (as a pair). The witness is $\mathsf{loop}(\mathsf{firstOfPair} \gg \mathsf{Put})$: the body extracts the input from the pair and writes it to the feedback channel.

Loop provides an independent witness of the same input-loss invariant exploited by fanout. It is not a qualitatively different impossibility — the root cause is identical — but it demonstrates that the obstruction is not specific to the fanout combinator. Any arrow primitive whose body requires access to the original input (alongside other channels) is subject to the same bind-based bottleneck.

Theorem 10 (No faithful direct-bind encoding preserves loop)**.** *For any* `BindEncoding3` (m, k) *where* m *satisfies Get-semantics* ($\mathsf{runMonad}_3(m,\ input,\ state) = (state,\ state)$) *and* k *satisfies Put-semantics* ($\forall x\, s.\ \mathsf{runMonad}_3(k\, x,\ x,\ s).2 = x$)*, if* $input \neq state$*:*

$$\mathsf{runArrow}_3(\mathsf{loop}\,\mathsf{body},\ input,\ state).2 \neq \mathsf{runMonad}_3(\mathsf{bind}\, m\, k,\ input,\ state).2.$$

(Mechanized: `no_faithful_bind_encoding_preserves_loop`*.)*

Proof. Same structure as fanout. loop gives the body *both* the input and the feedback channel (as a pair). bind gives the continuation only the previous output. By Get-semantics, m's output is *state*; by Put-semantics, k writes *state* to the feedback. The arrow writes *input*. They diverge when $input \neq state$. □

The Structural Root Cause

Theorem 11 (Impossibility is structural)**.** *The arrow && makes a branch depend on the original input:* $\mathsf{runArrow}(\&\&(\mathsf{ident}, \mathsf{ident}),\ input,\ _).1 = input$*. The* bind *continuation sees only the previous output, not the input. This is why the fanout (Theorem 3) and loop (Theorem 4) impossibility theorems hold, and why recovering full monadic expressiveness requires* ArrowApply*, which provides the dynamic application capability absent from the basic arrow interface. (Mechanized:* `impossibilityIsStructural`*.)*

Remark 12 (Relation to prior work)**.** VanDomelen et al. (2025) observed the gap *empirically* (freer monad continuations are opaque to static analysis). The fanout (Theorem 3) and loop (Theorem 4) impossibility theorems prove the gap is not merely observational but *semantic*: no faithful direct-bind encoding preserves arrow semantics. This strengthens the expressive-power asymmetry from "the inverse translation requires ArrowApply" (Lindley, Wadler, Yallop (2011)) to "no faithful direct-bind encoding of && or loop exists."

An Indistinguishability-Based Information Boundary

The impossibility theorems of Section 3 show *which* arrow primitives cannot be encoded by bind under the direct-bind discipline. This section formalizes *how much* structural information is lost in the standard monadic encoding, via an indistinguishability argument. The main substantive result is the indistinguishability theorem itself; the ordering and "ground truth" statements follow largely by construction from the definitions of the three representations. We state them for completeness and to frame the strictness result, which is the non-trivial part.

The Key Indistinguishability

The central fact is not stipulated but proved: the monadic encoding conflates fanout with sequencing.

Theorem 13 (Fanout and seq are indistinguishable under monadic encoding)**.**

$$\mathsf{encodeMonad}(\&\&(f, g)) = \mathsf{encodeMonad}(\mathsf{seq}(f, g)).$$

Both encode to $\mathsf{bind}(\mathsf{encodeMonad}(f))(\lambda_.\, \mathsf{encodeMonad}(g))$*. (Mechanized:* `encodeMonad_fanout_eq_seq`*, by* rfl*.)*

This is the formal version of the "false dependency" phenomenon: when arrow && is translated to bind, the second computation appears to depend on the first's result, destroying the independence information. No observer of the *encoded monadic representation* can distinguish $\&\&(f, g)$ from $\mathsf{seq}(f, g)$ — they encode to the same term. (The original arrow program obviously distinguishes them; the point is that this distinction is lost in the encoding.)

The Information Ordering

We define $\sigma(R, C) = \{f \mid f \text{ is recoverable from } R\text{'s encoding of } C\}$, where a structural fact f is *recoverable* from representation R iff there exists an observer O_R such that $O_R(\mathsf{encode}_R(C)) = \mathsf{holds}(f, C)$ for every C. That is: an observer of the encoded representation can determine whether f holds of the original computation.

Indistinguishability (Theorem 6) then implies non-recoverability directly: if $\mathsf{encode}_R(C_1) = \mathsf{encode}_R(C_2)$ but $\mathsf{holds}(f, C_1) \neq \mathsf{holds}(f, C_2)$, then no observer O_R can compute $\mathsf{holds}(f, \cdot)$ correctly on both. The three representations are:

- FA (freer arrow): explicit tree with nodes for seq, &&, left, loop, op, apply.
- FM (freer monad): explicit tree with nodes for bind, left, loop, op. By Theorem 6, && and seq encode to the same bind node.
- OM (opaque monad): a black-box function. No structure is inspectable.

Theorem 14 (The sigma measure-ordering)**.** *For any computation C:*

$$\sigma(\mathsf{OM}(C)) \subseteq \sigma(\mathsf{FM}(C)) \subseteq \sigma(\mathsf{FA}(C)).$$

This holds by construction: OM *exposes nothing,* FM *exposes the* bind *tree (which subsumes* OM*'s information), and* FA *exposes a strictly finer tree (with a dedicated* && *node). (Mechanized:* `sigma_ordering`*.)*

The non-trivial content is the *strictness* of the $\mathsf{FM} \subseteq \mathsf{FA}$ inclusion:

Theorem 15 (Strict inequality for fanout)**.** *For $C = \&\&(f, g)$:* fanout_independence $\in \sigma(\mathsf{FA}(C))$ *but* fanout_independence $\notin \sigma(\mathsf{FM}(C))$*. This follows from Theorem 6: since $\&\&(f, g)$ and $\mathsf{seq}(f, g)$ are indistinguishable under* FM*, any fact that distinguishes them (like* fanout_independence*) is not recoverable. (Mechanized:* `strict_fanout`*.)*

The $\mathsf{OM} \subseteq \mathsf{FM}$ inclusion is also strict for computations with inspectable structure (e.g., effect sets), but the $\mathsf{FM} \subseteq \mathsf{FA}$ strictness is the substantive result, as it rests on the indistinguishability theorem rather than on the definition of opacity alone.

ArrowApply Preserves sigma

Theorem 16 (ArrowApply preserves sigma)**.** $\sigma(\mathsf{FA{+}app}(C)) = \sigma(\mathsf{FA}(C))$*. The σ measure is defined over the original structural fact language (fanout independence, branching, feedback, etc.); the* app *constructor is a new node in the representation vocabulary but is not itself a structural fact that σ tracks. Since* app *is a visible tree node (not an opaque closure), it adds monadic power without introducing opacity, so no existing structural fact becomes unrecoverable. (Mechanized:* `arrowapply_preserves_sigma`*, by* rfl*;* `arrowapply_preserves_fanout` *for the fanout-specific corollary.)*

Non-Maximality

The statement that $\sigma(\mathsf{FA}, C) = \{f \mid \mathsf{holds}(f, C)\}$ holds by construction: we defined holds relative to the freer arrow representation, so FA recovers exactly the facts that hold. (Mechanized: `sigmaFA_is_ground_truth`.) This is a definitional fact, not a deep theorem; we state it to frame the non-maximality result.

Theorem 17 (Non-maximality)**.** *There exists a representation (*TaggedFreerArrow*: freer arrow with provenance labels on each* op*) with full monadic power (includes* app*) that recovers strictly more than* FA+app*: for C with ops,* op_provenance $\in \sigma(\mathsf{Tagged}, C)$ *but* op_provenance $\notin \sigma(\mathsf{FA{+}app}, C)$*. (Mechanized:* `maximality_is_false`*.)*

Remark 18 (What FAA achieves, and what it does not)**.** The freer arrow with ArrowApply achieves both (a) full monadic power (via app) and (b) full observability of the structural facts tracked by σ. It is *not* the maximal representation — richer representations (e.g., TaggedFreerArrow) can carry metadata (provenance, source locations, cost models) beyond the structural fact language, recovering strictly more. We do not claim minimality over all conceivable representations. We claim that FA+app *has* both properties and that $\sigma(\mathsf{FM})$ is a strict subset: the monadic representation loses fanout independence (Theorem 8), while FA+app does not (Theorem 9).

Representational Superiority

Syntactic Independence (Brief)

The standard arrow interface (Hughes (2000), Lindley, Wadler, and Yallop (2010)) uses three primitives (arr, $\gg$, first). We extend to five by adding left (sums/choice) and loop (feedback). In the free arrow calculus, each of the five primitives is syntactically non-derivable from the other four: left introduces Sum-handling, first introduces $\times$-handling, loop introduces feedback, and $\gg$/arr are trivially independent. (Mechanized in `Basic.lean` via type-inequality for distinct inductive constructors.)

This establishes *syntactic* non-derivability (distinct constructors have distinct types), not *semantic* independence. A semantic independence theorem would require, for each primitive, constructing a model satisfying the other four laws in which that primitive is not definable. We do not prove that here, and note that the syntactic result is close to definitional.

Expressive Equivalence, Representational Superiority

By Lindley, Wadler, and Yallop's result (2011), arrows equipped with ArrowApply are expressively equivalent to monads (ArrowApply $\cong$ Monad): both can represent the same computations. This is an external theorem about the arrow *interface*, not our own formalization. Our contribution is the *representation* result: the five-primitive freer arrow with ArrowApply carries the same computational content in an inspectable tree, while the freer monad carries it in an explicit bind tree with opaque continuations. The difference is *representational*:

Property	Freer arrow with ArrowApply	Freer monad
Expressive power	Full monadic (via app)	Full monadic
Representation	Inspectable tree	Explicit bind tree with opaque continuations
&& structure	Visible node	Lost (encodes to bind, Theorem 6)
left structure	Visible node	Visible node
loop structure	Visible node	Visible node
σ (Section 4)	Full structural facts	Strict subset (Theorem 8)

The impossibility theorems of Section 3 show *why* the representation matters: the arrow primitives that require input access (&&, loop) cannot be encoded by bind alone under the direct-bind discipline, so the monadic representation *cannot* carry their structure—it must be re-derived at runtime. The arrow representation carries it by construction.

> *The five-primitive freer arrow with* ArrowApply *is a representation achieving both full monadic power and full observability of the structural facts tracked by σ, while the freer monad achieves full monadic power but loses fanout independence.*

Empirical Corroboration

The impossibility and σ results are formal. We corroborate them with 242 Rust tests (all passing) that implement three representations of the same computations — freer arrow, opaque monad, freer monad — and check that the opaque monad is semantically adequate but structurally lossy: sequential composition is preserved, but fanout/independence, branching, effect sets, per-branch effects, dependency graphs, and feedback are all lost. The freer monad recovers everything, confirming the loss is about *opacity* (closures hide structure), not monadic structure per se. We do not present this as a systematic empirical study but as a sanity check that the formal asymmetry manifests in executable implementations. The test suite is in the supplementary material.

Honest limitation. Structural explicitness does not *require* arrows—one could define a disciplined lambda calculus with explicit let-binding for every subexpression. The arrow advantage is that the discipline is

enforced by the representation, not *imposed by convention*. Whether enforced discipline is an advantage depends on the use case; we state this as an architectural argument, not a formal result.

Related Work

Hughes (2000). Introduced arrows and stated the arrow laws (2000). Our $\mathcal{T}$ is the standard translation implicit in Hughes's presentation; we make linearity and compositionality explicit and mechanize them.

Lindley, Wadler, Yallop (2008). Established the bidirectional correspondence between the arrow calculus and lambda with products and sums (2010). Our $\mathcal{T}$ is an instance of their forward translation.

Lindley, Wadler, Yallop (2011). Established ArrowApply $\cong$ Monad and the strict inclusions idioms $\subset$ arrows $\subset$ monads (2011). Our contribution is the semantic impossibility (Section 3), which strengthens this from "the inverse translation requires ArrowApply" to "no faithful direct-bind encoding of && or loop exists."

Kiselyov and Ishii (2013). Freer monads (2015) make the same structural insight that underlies our representation asymmetry: explicit, traversable representations preserve information that opaque representations destroy. Our indistinguishability theorem (Section 4) formalizes this.

Lindley (2014). Generalized algebraic effects and effect handlers to support idiom and arrow computations (2014). Our work is complementary: we identify a concrete semantic obstruction (the bind continuation law) for specific combinators, rather than a general handler calculus.

VanDomelen et al. (2025). Introduced freer arrows and observed the observational gap empirically (2025). Our contribution is the formal, mechanized impossibility (Section 3) and the indistinguishability boundary (Section 4).

Conclusion

We have proved a scoped semantic impossibility: under a precisely defined class of direct-bind encodings (`BindEncoding`), no faithful encoding of arrow && or loop preserves arrow semantics. The root cause is a single encoding-independent law of bind (provable by rfl): the continuation is invoked with m's output, never the original input. Arrow combinators can preserve access to an earlier computation input across composition; monadic bind exposes only the preceding computation's result to its continuation. This is not a claim that monads cannot encode arrows — a monad with a side channel for the input can — but that no *direct-bind* encoding (one without such a side channel) suffices.

We formalized the information loss via an indistinguishability argument: encodeMonad(&&) = encodeMonad(seq) by rfl, so any observer of the encoded monadic representation cannot distinguish fanout from sequencing. We derived the ordering $\sigma(\mathsf{OM}(C)) \subseteq \sigma(\mathsf{FM}(C)) \subseteq \sigma(\mathsf{FA}(C))$, with the FM $\subseteq$ FA inclusion strict for fanout computations, proved ArrowApply preserves σ, and showed that maximality over all representations is false.

With ArrowApply, the five arrow primitives are expressively equivalent to monads (Lindley, Wadler, Yallop (2011)) but representationally superior: the same computational content is carried in an inspectable tree, not an opaque closure.

As a negative side-result, the standard arrow-to-lambda translation $\mathcal{T}$ is linear, compositional, and law-preserving. No translation asymmetry exists. The genuine asymmetry is semantic and representational, not translational.

Mechanization. The Lean 4 formalization comprises 13 modules with 108 theorems, all with zero `sorry`. The core modules are: `FanoutUnencodable.lean` (27 theorems: `BindEncoding` class, Get/Put semantics, encoding-independent bind law, universal fanout/loop unencodability, choice with weaker hypothesis), `StructuralInfo.lean` (13 theorems: indistinguishability, σ ordering, strict inequality, ArrowApply preservation, non-maximality), `Translation.lean` (22 theorems: linearity, compositionality, loop extension),

`Basic.lean` (7 theorems: syntactic independence classifications), `ProofRelevant.lean` (3 theorems: proof-relevant subject reduction), and `RefinementCharacterization.lean` (6 theorems). The remaining 7 modules (30 theorems) cover proof graphs, invariant domains, and structural inference.

Axiom dependencies. The formalization uses three sound axioms, all standard meta-theorems: (1) `BetaEta_confluence_normal_forms` — confluence of $\beta\eta$-reduction on STLC (Tait–Martin-Löf), stated as an axiom because the full confluence proof is beyond the scope of this formalization; (2) `Classical.choice` — the law of excluded middle / choice, used in `ProofRelevant.lean` to extract witnesses from non-emptiness proofs; (3) type inequality between distinct inductive constructors, used in `Basic.lean` for syntactic independence. No `sorry` appears in any proof. The formalization is supported by 242 Rust tests (all passing).

References

Appendix A: The Forward Translation — A Negative Result

As a negative side-result, we investigate whether the standard arrow-to-lambda translation $\mathcal{T}$ suffers size blowup or non-compositionality. We prove it does not. This addresses a conjectured translation asymmetry and clarifies that the genuine asymmetry is semantic (Section 3), not translational.

The Translation

We work in the simply-typed lambda calculus with products ($\times$), sums ($+$), and a fixed-point combinator fix, using de Bruijn indices. We call this calculus $\lambda_{\times,+,\mathsf{fix}}$. The standard translation $\mathcal{T} : \mathcal{A} \to \Lambda$ is:

$$\mathcal{T}(\mathsf{arr}\, f) = f \qquad \mathcal{T}(f \gg g) = \lambda x.\, \mathcal{T}(g)\,(\mathcal{T}(f)\,x)$$
$$\mathcal{T}(\mathsf{first}\, f) = \lambda(x, y).\, (\mathcal{T}(f)\,x,\, y)$$
$$\mathcal{T}(\mathsf{left}\, f) = \lambda s.\, \mathsf{case}\, s\, [\mathsf{inl}.\, \mathsf{inl}\, (\mathcal{T}(f)\,x) \mid \mathsf{inr}.\, \mathsf{inr}\, y]$$
$$\mathcal{T}(\mathsf{loop}\, f) = \lambda x.\, \mathsf{fst}\, (\mathsf{fix}\, (\lambda r.\, \mathcal{T}(f)\,(x, r)))$$

This is an instance of the Lindley–Wadler–Yallop forward translation (2010).

Summary: No Translation Asymmetry

Property	Status	Mechanized
Linearity ($\lvert\mathcal{T}(p)\rvert \leq 8\lvert p\rvert$)	Holds	`T_linear`
Compositionality of $\gg$	Definitional	`T_comp_is_composition`
Compositionality of first	Up to β	`T_first_compositional`
Compositionality of left	Up to $\beta\eta$ + commuting	`T_left_compositional`
Loop extension law	Up to $\beta\eta\mathsf{fix}$ (Bekić (1969))	`loopExt_eq_under_BetaEtaFix`
Loop law fails under $\beta\eta$	Existential	`loopExt_not_BetaEta`

The forward translation is linear, compositional, and law-preserving (up to the general recursion subtlety: the loop extension law holds under $\beta\eta$ plus fix-fusion/Bekić's lemma, but not under $\beta\eta$ alone, since fix is inert without unrolling). No translation asymmetry exists.

Appendix B: Choice Unencodability (Weaker Hypothesis)

The fanout and loop theorems of Section 3 derive divergence purely from Get/Put semantic specifications. We sketch here an analogous result for left (choice), noting that its hypothesis is weaker.

We extend the value domain to include sums ($\mathsf{Left}\, v$, $\mathsf{Right}\, v$) and define Arrow_2 with a choice combinator that branches on the input. The witness is $\mathsf{left}(\mathsf{Get},\ \mathsf{Pure}\, 42)$: with input $\mathsf{Right}\, 99$, the arrow branches to $\mathsf{Pure}\, 42$.

Theorem 19 (Choice unencodability (weaker hypothesis))**.** *For all monadic m satisfying Get-semantics and all k, if k applied to state does not produce the arrow result:*

$$\mathsf{runArrow}_2(\mathsf{left}(\mathsf{Get}, \mathsf{Pure}\, 42),\ \mathsf{Right}\, 99,\ \mathit{state}).1 \neq \mathsf{runMonad}_2(\mathsf{bind}\, m\, k,\ \mathsf{Right}\, 99,\ \mathit{state}).1.$$

(Mechanized: `no_bind_encoding_preserves_choice`.)

Remark 20 (Why this is weaker)**.** The choice theorem requires an additional hypothesis (`hBranch`) stating that k applied to the state does not produce the arrow result — close to assuming what we want to prove. A version with a clean Put-like constraint (analogous to fanout/loop) would require a richer value domain. We present this as supporting evidence that the same input-loss invariant applies to choice, not as an equally tight universal theorem.

Bekić, H. 1969. *Definable Operations in General Algebras, and the Correction-Free Abstract Calculus.* IBM Laboratory Vienna.

Hughes, J. 2000. "Generalising Monads to Arrows." *Science of Computer Programming* 37 (1–3): 67–111.

Kiselyov, O., and H. Ishii. 2015. "Freer Monads, More Extensible Effects." *Proc. ACM SIGPLAN Haskell Symposium*, 94–105.

Lindley, S. 2014. "Algebraic Effects and Effect Handlers for Idioms and Arrows." *Proc. ACM SIGPLAN Workshop on Generic Programming (WGP)*, 47–58.

Lindley, S., P. Wadler, and J. Yallop. 2010. "The Arrow Calculus." *Journal of Functional Programming* 20 (1): 51–77.

Lindley, S., P. Wadler, and J. Yallop. 2011. "Idioms Are Oblivious, Arrows Are Meticulous, Monads Are Promiscuous." *Electronic Notes in Theoretical Computer Science* 229 (5): 97–117.

VanDomelen, G., G. Shen, L. Kuper, and Y. Li. 2025. "Freer Arrows and Why You Need Them in Haskell." *Proc. ACM SIGPLAN Haskell Symposium*, 94–108.